CASE STUDY

Dijkstra's Shortest Path Algorithm

Submitted by

Akashdeep Singh (25MCA20157)

in partial fulfillment for the award of the degree of

Master of Computer Application

Chandigarh University

April , 2026

S.NO	Table of Content	Page No.
1	ACKNOWLEDGEMENT	
2	ANALYSIS / CRITICAL THINKING	
3	Main Concepts and Parameters	
4	Algorithm Description	
5	Algorithm Design & Representation	
6	Complexity Analysis	
7	Pseudocode & Implementation	
8	Optimization Techniques	
9	TECHNICAL IMPLEMENTATION	
10	References	

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty and guide for their continuous support and guidance throughout this case study. Their encouragement and insights have helped me understand algorithm design, time complexity analysis, and real-world applications of DAA concepts such as shortest path algorithms.

ANALYSIS / CRITICAL THINKING

The Shortest Path Optimization System is designed to efficiently determine the minimum distance between nodes in a network using graph algorithms. This system is widely applicable in real-world scenarios such as GPS navigation, network routing, and transportation systems.

The primary objective is to compute the shortest path from a source node to all other nodes in a weighted graph. This is achieved using Dijkstra's Algorithm, which ensures optimal path selection by minimizing total cost or distance.

The system must handle large datasets efficiently, requiring proper algorithm design and complexity optimization. Time complexity plays a crucial role, especially when dealing with dense graphs or real-time systems.

Data structures like priority queues (min-heaps) are used to improve performance. The algorithm ensures that once a node is visited with the shortest distance, it is not reprocessed, thus improving efficiency.

From a computational perspective, this system demonstrates how algorithm optimization reduces execution time and improves scalability in real-world applications.

1. Main Concepts and Parameters

The system revolves around graph-based components:

1. **Graph (G):**
   A graph is a structure of vertices and edges used to represent networks. It includes number of vertices (V), edges (E), and weights that show cost or distance between nodes.
2. **Vertex (Node):**
   A vertex represents a location or point in the graph. Each node has a unique ID, a distance value from the source, and a visited status to track processing.
3. **Edge:**
   An edge connects two vertices and represents a path between them. It contains source, destination, and weight (cost, distance, or time).
4. **Source Node:**
   It is the starting point of the shortest path algorithm. The distance of the source node is initialized to 0, while all other nodes are set to infinity.
5. **Distance Array:**
   This array stores the shortest known distance from the source node to every other vertex. It is updated whenever a shorter path is found.
6. **Priority Queue (Min Heap):**
   A data structure used to efficiently select the node with the smallest distance. It helps improve performance in algorithms like Dijkstra.
7. **Visited Set:**
   A collection of nodes that have already been processed. Once a node is visited, its shortest distance is finalized and it is not processed again.

2. Algorithm Description

Dijkstra's Algorithm is a **greedy algorithm** used to find the **shortest path from a source node to all other nodes** in a weighted graph. It works efficiently when all edge weights are **non-negative**.

- The algorithm starts from the **source node**, assigning its distance as **0**, while all other nodes are initialized with **infinity (∞)** to represent unknown distances.
- It uses a **priority queue (min heap)** to always select the node with the **smallest current distance**.
- At each step, the node with the **minimum distance** is selected and marked as **visited**, meaning its shortest path is finalized.
- Then, the algorithm checks all **adjacent (neighboring) nodes** of the selected node.
- If a shorter path to a neighbor is found (current distance + edge weight < previous distance), the **distance is updated**. This process is called **relaxation**.
- This process continues repeatedly, selecting the next minimum-distance node and updating its neighbors, until **all nodes are visited** or no shorter paths are possible.
- The final result is a set of **shortest distances from the source node to every other node**, and it can also be used to trace the actual shortest paths.
- Dijkstra's Algorithm guarantees **optimal results only when all edge weights are non-negative**, as negative weights can lead to incorrect calculations.

OBJ

3. Algorithm Design & Representation

Dijkstra's Algorithm can be represented using different graph structures and supporting data structures. One common method is the adjacency list representation, where each vertex stores a list of its neighboring vertices along with the weights of the connecting edges. This method is memory efficient and works best for sparse graphs where the number of edges is small. It also improves performance when used with a priority queue.

Another method is the adjacency matrix representation, where a two-dimensional matrix of size V × V is used to store edge weights between vertices. If there is no edge between two nodes, the value is stored as 0 or infinity. This representation is simple to understand and implement but consumes more memory, making it suitable mainly for dense graphs where the number of edges is high.

The graph itself is represented as G(V, E), where V denotes the set of vertices and E denotes the set of edges. Each edge connects two vertices and has an associated weight representing cost, distance, or time. This structure forms the base on which the algorithm operates.

A distance array is used to store the shortest known distance from the source node to all other vertices. Initially, the distance of the source node is set to 0, while all other nodes are assigned infinity to indicate that they are not yet reachable. As the algorithm progresses, this array is updated whenever a shorter path is found.

A priority queue (min heap) is used to efficiently select the vertex with the smallest distance at each step. It always processes the node with the minimum value first, which helps in reducing the overall time complexity and ensures faster execution of the algorithm.

For example, in a graph with vertices A, B, and C, where A is connected to B with weight 2 and to C with weight 5, the adjacency list representation would

be A → (B,2), (C,5). Initially, the distance array would be set as A = 0, B = ∞, and C = ∞, and it would be updated as the algorithm proceeds.

Overall, adjacency list representation is preferred for better efficiency in sparse graphs, while adjacency matrix is simpler but consumes more space. The combination of graph representation, distance array, and priority queue forms the core of Dijkstra's Algorithm and helps in finding the shortest path efficiently.

4. Complexity Analysis

The performance of Dijkstra's Algorithm mainly depends on the data structures used to store the graph and to select the minimum distance node. When a simple array is used, the algorithm checks all vertices to find the minimum distance node at each step. This results in a time complexity of **$O(V^2)$**, where V is the number of vertices. This approach is easy to implement but becomes inefficient for large graphs because it repeatedly scans all nodes.

When a **min heap or priority queue** is used, the efficiency improves significantly. Instead of checking all vertices, the algorithm directly extracts the node with the smallest distance in **log V time**. Along with this, updating distances (relaxation) also becomes efficient. In this case, the overall time complexity becomes **$O((V + E) \log V)$**, where E is the number of edges. This makes the algorithm much faster, especially for sparse graphs where the number of edges is relatively small compared to vertices.

The **space complexity** of Dijkstra's Algorithm is **$O(V + E)$** because it stores the graph (using adjacency list or matrix), a distance array of size V, and additional structures like the priority queue and visited set. The adjacency list typically takes $O(V + E)$ space, while other components require $O(V)$ space.

Overall, using a priority queue greatly improves the performance of Dijkstra's Algorithm compared to a simple array. It reduces unnecessary computations and makes the algorithm suitable for large-scale graphs, ensuring faster and more efficient shortest path calculations.

5. Pseudocode & Implementation

Pseudocode:

```
Initialize distance array dist[] with infinity
dist[source] = 0

Create priority queue pq
Insert source node into pq

While pq is not empty:
    Extract node u with minimum distance
    For each neighbor v of u:
        If dist[u] + weight(u, v) < dist[v]:
            Update dist[v]
            Insert v into pq
```

Algorithm Steps

Write this below pseudocode:

1. Initialize all distances as infinity except source node.
2. Insert source node into priority queue.
3. Extract the node with minimum distance.
4. Update distances of adjacent nodes (relaxation step).
5. Repeat until all nodes are processed.
6. Store final shortest distances.

Sample Code

```
#include <bits/stdc++.h>
using namespace std;

void dijkstra(int V, vector<pair<int,int>> adj[], int src) {
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    vector<int> dist(V, INT_MAX);

    dist[src] = 0;
    pq.push({0, src});

    while(!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        for(auto it : adj[u]) {
            int v = it.first;
            int weight = it.second;

            if(dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }
}
```

6. Optimization Techniques

To improve the efficiency of Dijkstra's Algorithm, several optimizations can be applied. One important improvement is the use of a **min heap or priority queue**, which helps in quickly selecting the node with the minimum distance instead of scanning all vertices. This significantly reduces the time complexity and makes the algorithm suitable for large graphs.

Another optimization is to **avoid reprocessing nodes** by using a visited array or set. Once a node is marked as visited, its shortest distance is finalized and it does not need to be processed again. This prevents unnecessary computations and improves overall performance.

Using an **efficient graph representation** also plays a crucial role. An adjacency list is generally preferred over an adjacency matrix because it consumes less memory and allows faster traversal of neighbors, especially in sparse graphs where the number of edges is limited.

Additionally, **early termination** can be applied when the goal is to find the shortest path to a specific destination rather than all nodes. In such cases, the algorithm can stop as soon as the destination node is reached with the minimum distance, saving time and computational effort.

Overall, these optimizations reduce unnecessary operations, improve speed, and make Dijkstra's Algorithm more efficient and practical for real-world applications.

PROJECT DESIGN AND IMPLEMENTATION / TECHNICAL IMPLEMENTATION

The system architecture includes:

1. **Input Layer:**
   Accepts graph data (nodes, edges, weights)
2. **Processing Layer:**
   Implements Dijkstra's Algorithm using priority queue
3. **Output Layer:**
   Displays shortest path and minimum distance

Programming Languages Used:

- C++
- Java
- Python

Libraries:

- STL (priority_queue)
- Graph libraries

REFERENCES

1. • **Cormen, T. H., et al. – *Introduction to Algorithms (MIT Press)*:**
   A standard textbook (CLRS) that provides detailed theory and analysis of algorithms including Dijkstra's Algorithm.

2. • **Kleinberg, J., Tardos, É. – *Algorithm Design*:**
   Explains algorithm design techniques with clear examples, including greedy algorithms and shortest path problems.

3. • **GeeksforGeeks – Dijkstra Algorithm:**
   An online resource with easy explanations, code examples, and step-by-step working of the algorithm.

4. • **TutorialsPoint – Graph Algorithms:**
   Provides simple tutorials and practical understanding of graph algorithms including Dijkstra's Algorithm.

5. • **CLRS Algorithm Book:**
   A widely used academic reference covering algorithm concepts, complexity, and real-world applications.

6. • **Research Papers on Shortest Path Algorithms:**
   Provide advanced knowledge, improvements, and variations of shortest path techniques used in real-world systems.

7. • **GitHub Repositories on Graph Algorithms:**
   Contain practical implementations of Dijkstra's Algorithm in different programming languages for better understanding.

8. • **Online Resources on Optimization Techniques:**
   Include blogs, articles, and tutorials that explain ways to improve algorithm efficiency and performance.